

Relational Graph-Aware Schema Retrieval for Robust Text-to-SQL over Complex Enterprise Databases

Applicant: Muhammad Maaz Khan

Undergraduate Degree: BS Computer Science, UET Peshawar

Target Degree: Master of Science (MS) in Computer Science

Prospective Supervisor: Prof. Huanhuan Chen (IEEE Fellow)

Host Institution: School of Computer Science & Technology, USTC

Target Funding: CAS-ANSO Scholarship for Young Talents

EXECUTIVE ABSTRACT

Natural Language Interfaces to Databases (NLIDB) translate conversational user questions into executable SQL queries. However, enterprise databases with dozens to hundreds of interconnected tables pose severe schema-linking bottlenecks for Large Language Models (LLMs). Conventional Retrieval-Augmented Generation (RAG) models schemas as independent text blocks, retrieving tables through semantic vector similarity. Over multi-table relational structures, vector retrieval frequently retrieves terminal tables while omitting essential intermediate junction tables, triggering invalid joins, column hallucinations, and execution failures. This proposal presents **Relational Graph-RAG**, a framework that models database metadata as a directed relational knowledge graph. By executing structural path traversals across foreign-key dependencies, the system isolates the minimal connected subgraph required to bridge user entities, supplying LLMs with structurally sound, hallucination-resistant context. Evaluated against direct prompting and vector-RAG baselines on public benchmarks (Spider, BIRD), this research investigates the trade-offs between execution accuracy, token efficiency, and schema scalability.

1. Introduction & The Core Problem

Modern enterprise databases (ERP, CRM, and financial data lakes) enforce relational integrity through normalized schemas comprising dozens to hundreds of tables linked by primary and foreign keys. When a user asks an analytical question such as *"What were total electronics sales purchased by Computer Science students in 2025?"*, answering requires constructing a multi-table SQL query joining five distinct entities: departments → users → orders → products → categories.

Standard Large Language Models encounter severe degradation on such queries. If the entire database schema is provided in the prompt, context windows quickly saturate, incurring prohibitive token costs and triggering *"Lost-in-the-Middle"* attention decay. Conversely, if naive semantic vector retrieval is employed, the retriever selects tables that match query keywords (e.g., departments and products) but omits the bridging junction tables (users and orders). Consequently, the LLM hallucinates non-existent foreign keys or produces syntactically broken queries.

2. Relational Graph-RAG System Architecture

Rather than treating schemas as collections of unstructured text documents, **Relational Graph-RAG** formalizes the database as a directed relational knowledge graph $G = (V, E)$. Vertices V represent tables and attribute columns; directed edges E represent primary-foreign key relationships, constraints, and cardinalities.

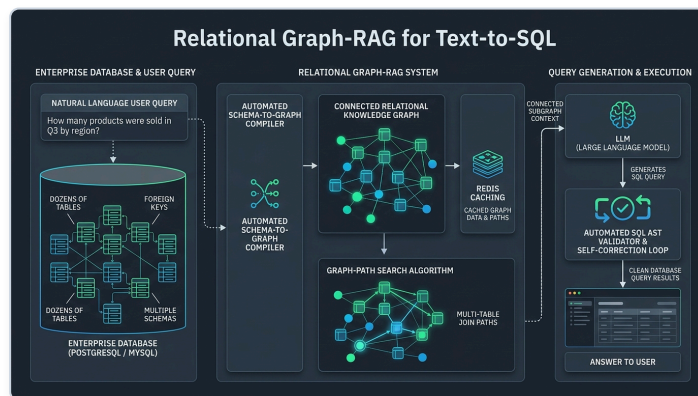


Figure 1: End-to-end Relational Graph-RAG architecture showing automated schema graph compilation, multi-hop path search, Redis subgraph caching, and closed-loop SQL AST verification.

The system executes a four-stage query lifecycle:

- 1. Automated Schema Mining:** An automated ingestion daemon queries PostgreSQL/MySQL metadata (information_schema.tables, columns, key_column_usage) to dynamically construct the relational graph without manual tagging.
- 2. Multi-Hop Path Traversal:** An entity extractor maps natural language query concepts to graph nodes. A shortest-path graph search (evaluating Dijkstra and Steiner tree algorithms) identifies the minimal connected join subgraph connecting all target entities.

3. **Sub-Millisecond Redis Caching:** Frequently accessed schema paths (e.g., orders → payments → users) are cached in Redis to eliminate redundant graph traversal latencies during peak analytical loads.
4. **Closed-Loop SQL AST Validation:** Candidate SQL emitted by the LLM is parsed using an Abstract Syntax Tree (AST) validator (sqlglot) before database execution. If an invalid column or unconstrained join is detected, structured diagnostic feedback is fed back to the model for zero-shot self-correction.

3. Controlled 3-Stage Experimental Methodology

To establish empirical academic rigor, the framework will be benchmarked through a controlled comparative evaluation across three distinct systems on standardized public datasets (Spider cross-domain benchmark and BIRD enterprise database benchmark):

Experimental Stage	Context Generation Mechanism	Input Token Footprint	Hypothesized Failure Mode
Stage 1: Direct Prompting	Full raw schema dump (DDL) passed in prompt	~35,000 – 50,000 tokens	High token cost, context window saturation, attention decay
Stage 2: Standard Vector RAG	Dense vector embeddings (top-k semantic similarity)	~6,000 – 8,000 tokens	Omits intermediate junction tables; broken multi-table joins
Stage 3: Relational Graph-RAG	Minimal connected subgraph with explicit join constraints	~2,500 – 4,000 tokens	Hypothesis: Eliminates hallucinations, achieves highest execution accuracy

4. Quantitative Evaluation Metrics

The evaluation will measure six precise quantitative metrics:

- Execution Accuracy (EX %):** Percentage of generated queries that execute and return exact ground-truth tabular results.
- Schema-Linking Recall (SL %):** Proportion of ground-truth tables, columns, and foreign keys correctly identified.
- Valid SQL Compilation Rate (VSR %):** Proportion of generated queries that parse and compile with zero syntax errors.
- Token Consumption Efficiency:** Total prompt token overhead compared against baseline schema prompting.
- End-to-End Query Latency (P50/P99):** Processing time encompassing retrieval, generation, AST validation, and execution.
- Schema Scale Robustness:** Degradation profile as database schema scales from 10 → 50 → 200 tables.

5. 24-Month Master's Research & Academic Roadmap

Academic Phase	Core Research Activities	Milestones & Deliverables
Year 1: M1 – M3	Core graduate coursework; systematic literature review on schema linking; scientific Python & PyTorch training.	Course completion; formal literature review document.
Year 1: M4 – M6	Build automated PostgreSQL/MySQL metadata extractor; configure Spider/BIRD benchmark testbeds.	Reproducible Baseline A & B implementation.
Year 1: M7 – M9	Implement NetworkX schema graph compiler and multi-hop Steiner tree search heuristics.	Working Relational Graph-RAG prototype.
Year 1: M10 – M12	Execute preliminary comparative benchmarks; analyze schema-linking recall and latency profiles.	Annual laboratory review presentation; refined methodology.
Year 2: M13 – M15	Integrate SQL AST self-correction loop; test hybrid retrieval (dense embedding + graph search).	Closed-loop verification pipeline verified.
Year 2: M16 – M18	Conduct full ablation studies (varying schema size, graph search vs vector search); error taxonomy.	Complete experimental result corpus & graphs.
Year 2: M19 – M21	Compile thesis chapters (Introduction, Related Work, Architecture, Experiments, Discussion).	Full Master's thesis draft; conference submission if warranted.
Year 2: M22 – M24	Thesis revision based on committee review; final defense; experimental codebase archiving.	Master's Thesis Defense & Degree Completion.

6. Candidate Background, Feasibility, & Supervision Fit

Preparation & Practical Systems Readiness: I hold a Bachelor of Science in Computer Science from the University of Engineering and Technology (UET), Peshawar, Pakistan. Over the past 4+ years, I have worked professionally as a backend and full-stack software engineer architecting production databases (PostgreSQL, MySQL), distributed caching layers (Redis), and containerized microservices (Docker). While I do not possess prior academic publications, my daily experience designing relational schemas, writing complex multi-table joins, and debugging database bottlenecks provides immediate practical readiness to construct database extractors and experimental testbeds from Day 1. I am committed to dedicating Year 1 coursework to mastering theoretical machine learning and empirical research methodologies.

Alignment with Prof. Huanhuan Chen's Group: Prof. Huanhuan Chen's distinguished scholarship in Machine Learning, Data Mining, and Big Knowledge Engineering at USTC provides the ideal scientific foundation for this investigation. Framing relational schemas as structured knowledge graphs directly operationalizes knowledge engineering principles. Under Prof. Chen's mentorship, I aim to bridge applied database engineering with rigorous knowledge representation research.

7. Key Research Questions & Anticipated Contributions

The thesis will formally answer three primary research questions:

- **RQ1:** Does relational-graph-aware schema retrieval significantly improve Execution Accuracy (EX) over multi-table queries compared to standard vector retrieval baselines?
- **RQ2:** By what factor does subgraph context pruning reduce token consumption and generation latency across varying schema sizes?
- **RQ3:** How effectively does an AST-guided verification loop detect and self-repair relational join errors prior to database execution?

Expected Contributions: (1) An open-source, reproducible Relational Graph-RAG schema extraction and retrieval pipeline; (2) A comprehensive benchmark evaluation across varying enterprise schema complexities (10 to 200 tables); and (3) An empirically validated Master's thesis advancing natural language database interfaces.

8. Selected Foundational References

- [1] Yu, T., et al. "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL." *EMNLP 2018*.
- [2] Li, J., et al. "Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQL (BIRD)." *NeurIPS 2023*.
- [3] Gao, D., et al. "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation." *VLDB 2024*.
- [4] Pourreza, M., & Rafiei, D. "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction." *NeurIPS 2023*.
- [5] Chen, H., et al. "Knowledge graph quality control: A survey." *IEEE Transactions on Knowledge and Data Engineering*.